

INSTITUTO FEDERAL GOIANO – CAMPUS MORRINHOS - GO
TECNOLOGIA EM SISTEMAS PARA INTERNET

EGLEICE AMORIM, GEOVANA DANTAS E SARAH NECO

PROJETO ORDENAÇÃO

MORRINHOS – GO

2026

PROJETO ORDENAÇÃO

.Trabalho apresentado à disciplina de Estrutura de Dados II, do curso de Tecnologia em Sistemas para Internet do Instituto Federal Goiano – Campus Morrinhos, como requisito para avaliação da disciplina.

Professor: Fernando Matos Barbosa.

MORRINHOS – GO

2026

Sumário

COMPARAÇÃO DE ALGORITMOS DE ORDENAÇÃO: UMA ANÁLISE PRÁTICA DE DESEMPENHO UTILIZANDO JAVA.....	4
RESUMO.....	4
1 INTRODUÇÃO.....	4
2 OBJETIVOS.....	5
2.1 Objetivo Geral.....	5
2.2 Objetivos Específicos.....	5
3 REFERENCIAL TEÓRICO.....	5
3.1 Bubble Sort.....	6
3.2 Insertion Sort.....	6
3.3 Selection Sort.....	6
3.4 Comb Sort.....	7
3.5 Shell Sort.....	7
3.6 Quick Sort.....	7
3.7 Merge Sort.....	7
3.8 Tim Sort.....	8
3.9 Radix Sort.....	8
3.10 Bucket Sort.....	8
4 METODOLOGIA.....	9
5 RESULTADOS.....	10
5.1 Complexidade Teórica.....	10
5.2 Análise das Comparações.....	11
5.3 Análise das Trocas.....	12
5.4 Discussão dos Resultados.....	13
6 CONCLUSÃO.....	13
7 REFERÊNCIAS.....	14

COMPARAÇÃO DE ALGORITMOS DE ORDENAÇÃO: UMA ANÁLISE PRÁTICA DE DESEMPENHO UTILIZANDO JAVA

RESUMO

A ordenação de dados constitui uma das operações fundamentais da Ciência da Computação, sendo amplamente utilizada em sistemas computacionais, bancos de dados, aplicações web e sistemas corporativos. O desempenho de um algoritmo de ordenação influencia diretamente a eficiência do processamento de informações, especialmente em cenários que envolvem grandes volumes de dados. Neste trabalho foi desenvolvido um sistema em linguagem Java para implementação, execução e comparação de diferentes algoritmos de ordenação. Foram analisados os algoritmos Bubble Sort, Insertion Sort, Selection Sort, Comb Sort, Shell Sort, Quick Sort, Merge Sort, Tim Sort, Radix Sort e Bucket Sort. Os experimentos foram realizados com vetores contendo de 5 até 500.000 elementos, sendo avaliadas métricas como número de comparações, número de trocas e complexidade computacional. Os resultados obtidos permitiram identificar as diferenças de desempenho entre algoritmos quadráticos, algoritmos baseados em divisão e conquista e algoritmos não comparativos. Os resultados demonstram que algoritmos modernos apresentam vantagens significativas em conjuntos de dados de grande escala.

Palavras-chave: Algoritmos de Ordenação. Java. Estrutura de Dados. Complexidade Computacional. Desempenho.

1 INTRODUÇÃO

A organização eficiente de dados é uma necessidade constante em sistemas computacionais modernos. A ordenação de informações permite acelerar buscas, melhorar consultas em bancos de dados e otimizar diversas operações realizadas por softwares.

Ao longo da evolução da computação, diversos algoritmos de ordenação foram desenvolvidos, cada um apresentando características próprias relacionadas ao desempenho, consumo de memória e complexidade computacional.

A escolha inadequada de um algoritmo pode resultar em desperdício de recursos computacionais e aumento significativo do tempo de processamento. Dessa forma, torna-se importante compreender o comportamento de cada técnica de ordenação em diferentes cenários.

Este trabalho teve como objetivo implementar e comparar diferentes algoritmos de ordenação utilizando a linguagem Java, analisando quantitativamente o número de comparações realizadas, a quantidade de trocas efetuadas e o comportamento teórico de cada algoritmo.

2 OBJETIVOS

2.1 Objetivo Geral

Comparar o desempenho de diferentes algoritmos de ordenação através da implementação prática em Java e análise de métricas computacionais.

2.2 Objetivos Específicos

- Implementar diferentes algoritmos de ordenação;
- Medir o número de comparações realizadas;
- Medir o número de trocas efetuadas;
- Comparar o comportamento dos algoritmos em diferentes volumes de dados;
- Relacionar os resultados obtidos com a complexidade teórica dos algoritmos;
- Identificar quais algoritmos apresentam melhor desempenho para grandes conjuntos de dados.

3 REFERENCIAL TEÓRICO

Os algoritmos de ordenação podem ser classificados em algoritmos baseados em comparação e algoritmos não comparativos.

Os algoritmos baseados em comparação determinam a ordem dos elementos comparando pares de valores. Já os algoritmos não comparativos utilizam características específicas dos dados, como seus dígitos ou distribuição numérica.

3.1 Bubble Sort

O Bubble Sort é um dos algoritmos mais simples da computação. Seu funcionamento consiste em comparar elementos adjacentes e realizar trocas sempre que necessário.

Apesar de sua simplicidade, seu desempenho torna-se inadequado para grandes conjuntos de dados.

Complexidade:

- Melhor caso: $O(n)$
- Caso médio: $O(n^2)$
- Pior caso: $O(n^2)$

3.2 Insertion Sort

O Insertion Sort insere gradualmente cada elemento na posição correta dentro da porção já ordenada do vetor.

É eficiente para vetores pequenos e parcialmente ordenados.

Complexidade:

- Melhor caso: $O(n)$
- Caso médio: $O(n^2)$
- Pior caso: $O(n^2)$

3.3 Selection Sort

O Selection Sort busca o menor elemento do vetor e o posiciona na posição correta a cada iteração.

Complexidade:

- Melhor caso: $O(n^2)$
- Caso médio: $O(n^2)$

- Pior caso: $O(n^2)$

3.4 Comb Sort

O Comb Sort foi desenvolvido como uma melhoria do Bubble Sort. Sua principal característica é a utilização de um intervalo variável denominado gap.

Complexidade:

- Melhor caso: Aproximadamente $O(n \log n)$
- Caso médio: $O(n^2)$
- Pior caso: $O(n^2)$

3.5 Shell Sort

O Shell Sort reduz gradualmente os intervalos entre elementos comparados, aumentando a eficiência do Insertion Sort.

Complexidade:

- Melhor caso: aproximadamente $O(n \log n)$
- Caso médio: depende da sequência de intervalos (gaps) utilizada
- Pior caso: $O(n^2)$

A complexidade do Shell Sort não é única e depende diretamente da sequência de gaps adotada na implementação.

3.6 Quick Sort

O Quick Sort utiliza a estratégia de divisão e conquista, selecionando um pivô para particionar os dados.

Complexidade:

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n^2)$

3.7 Merge Sort

O Merge Sort divide recursivamente os dados em subconjuntos menores e posteriormente realiza sua fusão ordenada.

Complexidade:

- Melhor caso: $O(n \log n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n \log n)$

3.8 Tim Sort

O Tim Sort combina características do Insertion Sort e do Merge Sort, sendo utilizado internamente em diversas linguagens modernas.

Complexidade:

- Melhor caso: $O(n)$
- Caso médio: $O(n \log n)$
- Pior caso: $O(n \log n)$

3.9 Radix Sort

O Radix Sort ordena números analisando seus dígitos individualmente.

Complexidade:

- Melhor caso: $O(nk)$
- Caso médio: $O(nk)$
- Pior caso: $O(nk)$

3.10 Bucket Sort

O Bucket Sort distribui os elementos em grupos chamados buckets, que posteriormente são ordenados individualmente.

Complexidade:

- Melhor caso: $O(n + k)$
- Caso médio: $O(n + k)$
- Pior caso: $O(n^2)$

O desempenho do Bucket Sort depende da distribuição dos dados entre os buckets. Quando os elementos são distribuídos de forma equilibrada, o algoritmo apresenta desempenho próximo ao linear. Entretanto, se muitos elementos forem concentrados em um único bucket, o desempenho pode degradar para $O(n^2)$.

4 METODOLOGIA

O sistema foi desenvolvido em linguagem Java utilizando conceitos estudados na disciplina Estrutura de Dados II.

Foi criada uma aplicação capaz de:

- Ler arquivos contendo conjuntos de dados;
- Executar múltiplos algoritmos de ordenação;
- Registrar quantidade de comparações;
- Registrar quantidade de trocas;
- Medir o tempo de execução;
- Registrar os resultados obtidos em tabelas contendo as métricas de comparações, trocas e tempo de execução, possibilitando a análise comparativa entre os algoritmos estudados.

Os conjuntos de dados utilizados continham:

- 5 elementos;
- 100 elementos;
- 1.000 elementos;
- 10.000 elementos;
- 50.000 elementos;
- 100.000 elementos;
- 500.000 elementos.

Para evitar tempos excessivos de processamento, alguns algoritmos de complexidade quadrática não foram executados para os maiores conjuntos de dados.

Durante o desenvolvimento foram utilizadas ferramentas de Inteligência Artificial exclusivamente como apoio à revisão textual, correção gramatical, identificação de possíveis erros de programação e organização do relatório. O desenvolvimento do sistema, a implementação dos algoritmos, a execução dos experimentos, a análise dos resultados e as conclusões apresentadas neste trabalho foram realizados integralmente pelas autoras.

Os experimentos foram realizados utilizando a linguagem Java para implementação dos algoritmos de ordenação. O desenvolvimento ocorreu em ambiente compatível com Java, sendo executados testes com diferentes tamanhos de conjuntos de dados para análise do desempenho dos algoritmos.

Para cada execução foram registradas métricas de comparações, trocas e tempo de processamento, permitindo a comparação prática entre os métodos estudados.

Os experimentos foram realizados em um notebook ASUS Vivobook Go E1504FA, equipado com processador AMD Ryzen 5 7520U com Radeon Graphics (4 núcleos e 8 threads, frequência de 2,80 GHz), 8 GB de memória RAM e sistema operacional Microsoft Windows 11 Pro 64 bits (compilação 26200). O desenvolvimento e a execução dos algoritmos foram realizados em ambiente compatível com a plataforma Java.

5 RESULTADOS

5.1 Complexidade Teórica

A análise teórica dos algoritmos estudados permitiu compreender o comportamento esperado de cada método diante do aumento da quantidade de dados processados. Observa-se que Bubble Sort, Insertion Sort e Selection Sort apresentam complexidade quadrática na maioria dos cenários, tornando-se menos eficientes para grandes conjuntos de dados. Em contrapartida, algoritmos como Quick Sort, Merge Sort e Tim Sort apresentam crescimento logarítmico, proporcionando melhor desempenho em aplicações que exigem maior capacidade de processamento.

Tabela Teórica ▾			
ALGORITMO ▾	MELHOR CASO ▾	CASO MÉDIO ▾	PIOR CASO ▾
BUBBLE SORT	$O(n)$	$O(n^2)$	$O(n^2)$
INSERTION SORT	$O(n)$	$O(n^2)$	$O(n^2)$
SELECTION SORT	$O(n^2)$	$O(n^2)$	$O(n^2)$
COMB SORT	$O(n \log n)$	$O(n^2)$	$O(n^2)$
SHELL SORT	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$
QUICK SORT	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
MERGE SORT	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
TIM SORT	$o(n)$	$O(n \log n)$	$O(n \log n)$
RADIX SORT	$O(n-k)$	$O(n-k)$	$O(n-k)$
BUCKET SORT	$O(n+k)$	$O(n+k)$	$O(n+k)$

5.2 Análise das Comparações

Os resultados experimentais referentes ao número de comparações realizadas confirmam as previsões apresentadas pela análise teórica. Os algoritmos Bubble Sort e Selection Sort apresentaram crescimento expressivo na quantidade de comparações à medida que o tamanho dos conjuntos de dados aumentou.

O Insertion Sort apresentou comportamento semelhante, embora tenha realizado menor quantidade de comparações em alguns cenários específicos. Os algoritmos Quick Sort, Merge Sort e Tim Sort demonstraram crescimento significativamente mais controlado, mantendo desempenho superior mesmo para conjuntos de dados maiores.

Entre os algoritmos não comparativos, o Radix Sort apresentou crescimento próximo ao linear em relação ao tamanho dos dados processados, enquanto o Bucket Sort apresentou resultados satisfatórios em conjuntos menores, porém demonstrou aumento progressivo do custo computacional conforme o crescimento dos conjuntos analisados.

De modo geral, observou-se que os algoritmos com complexidade quadrática tendem a apresentar aumento acelerado do número de comparações à medida que a quantidade de elementos cresce, enquanto algoritmos mais eficientes mantêm comportamento mais estável e adequado para aplicações que demandam maior desempenho.

Tabela Comparações							
ALGORITMO	5 ELEMENTOS	100 ELEMENTOS	1000 ELEMENTOS	10 MIL ELEMENTOS	50 MIL ELEMENTOS	100 MIL EL	500 MIL EL
Bubble Sort	10	4950	499500	49995000	PAUSADO	PAUSADO	PAUSADO
Insertion Sort	7	2517	250745	25263587	PAUSADO	PAUSADO	PAUSADO
Selection Sort	10	4950	499500	49995000	PAUSADO	PAUSADO	PAUSADO
Comb Sort	13	1201	22709	326734	2083404	3966742	24833420
Shell Sort	9	932	14902	266804	1988803	4291824	28156289
Quick Sort	6	578	11959	150431	978047	1975862	13696593
Merge Sort	7	548	8704	120399	718115	1536390	8837069
Tim Sort	7	954	13591	172147	975492	2050677	11270776
Radix Sort	25	500	5000	50000	250000	500000	2500000
Bucket Sort	7	388	12104	262059	3790471	8378796	87591830

5.3 Análise das Trocas

Em relação à quantidade de trocas realizadas durante o processo de ordenação, os algoritmos Bubble Sort e Insertion Sort apresentaram os maiores valores entre os métodos analisados, evidenciando as limitações desses algoritmos em aplicações que envolvem grandes volumes de dados.

O Selection Sort destacou-se por realizar quantidade significativamente menor de trocas quando comparado aos demais algoritmos quadráticos, característica decorrente de sua estratégia de posicionar diretamente o menor elemento em cada etapa da ordenação.

Os algoritmos Quick Sort, Merge Sort e Tim Sort apresentaram crescimento mais moderado no número de trocas, demonstrando maior eficiência prática. O Radix Sort manteve comportamento compatível com sua complexidade teórica, enquanto o Bucket Sort apresentou desempenho dependente da distribuição dos elementos entre os buckets, variando conforme as características dos conjuntos analisados.

Os resultados obtidos demonstram que a quantidade de trocas está diretamente relacionada à estratégia adotada por cada algoritmo. Dessa forma, algoritmos que minimizam movimentações desnecessárias tendem a apresentar melhor desempenho, especialmente quando aplicados a conjuntos de dados extensos.

Tabela de trocas								
ALGORITMO	5 ELEMENTOS	100 ELEMENTOS	1000 ELEMENTOS	10 MIL ELEMENTOS	50 MIL ELEMENTOS	100 MIL ELEMENTOS	500 MIL ELEMENTOS	
Bubble Sort	5	2425	249754	25253594	PAUSADO	PAUSADO	PAUSADO	
Insertion Sort	5	2425	249754	25253594	PAUSADO	PAUSADO	PAUSADO	
Selection Sort	3	95	991	9990	PAUSADO	PAUSADO	PAUSADO	
Comb Sort	3	277	4450	63341	372470	753375	3967486	
Shell Sort	3	481	7407	151918	1314291	2843697	19912935	
Quick Sort	5	417	6497	76601	523239	1095693	8544298	
Merge Sort	4	274	4332	59277	354989	759772	4390146	
Tim Sort	5	754	10178	116963	644401	1338364	7301851	
Radix Sort	25	500	5000	50000	250000	500000	2500000	
Bucket Sort	5	100	1000	10000	50000	100000	500000	

5.4 Discussão dos Resultados

A comparação entre os resultados teóricos e experimentais demonstrou forte compatibilidade entre as previsões de complexidade e o comportamento observado durante os testes realizados. Os algoritmos de complexidade quadrática apresentaram crescimento acelerado tanto no número de comparações quanto na quantidade de trocas, tornando-se inviáveis para conjuntos de dados muito extensos.

Por outro lado, os algoritmos baseados na estratégia de divisão e conquista, como Quick Sort e Merge Sort, mantiveram desempenho significativamente superior, apresentando melhor escalabilidade conforme o crescimento dos conjuntos analisados. O Tim Sort também demonstrou excelente desempenho, confirmando sua ampla utilização em bibliotecas modernas de ordenação.

Entre os algoritmos não comparativos, o Radix Sort destacou-se como uma das soluções mais eficientes para conjuntos numéricos de grande porte, apresentando crescimento linear e baixo custo computacional. Os resultados obtidos reforçam a importância da escolha adequada do algoritmo de ordenação de acordo com as características dos dados e os requisitos de desempenho da aplicação.

6 CONCLUSÃO

O presente trabalho permitiu analisar o comportamento de diferentes algoritmos de ordenação por meio de experimentação prática e estudo teórico. A implementação dos algoritmos em linguagem Java possibilitou a comparação de métricas relevantes, como

número de comparações, quantidade de trocas e tempo de execução em diferentes volumes de dados.

Os resultados demonstraram que a escolha do algoritmo influencia diretamente a eficiência de uma aplicação computacional. Algoritmos de complexidade quadrática, como Bubble Sort, Insertion Sort e Selection Sort, apresentaram desempenho satisfatório apenas para conjuntos reduzidos de dados, tornando-se menos eficientes à medida que o volume de informações aumentou.

Por outro lado, algoritmos como Quick Sort, Merge Sort e Tim Sort apresentaram melhor escalabilidade e desempenho consistente, demonstrando maior adequação para aplicações que exigem processamento de grandes quantidades de dados.

Entre os algoritmos não comparativos analisados, os resultados indicaram que o Radix Sort apresentou excelente desempenho para conjuntos numéricos de grande porte, corroborando sua complexidade teórica linear em função da quantidade de elementos e dos dígitos processados.

Além da análise teórica, os resultados experimentais permitiram validar na prática os comportamentos previstos pela literatura especializada. A utilização de métricas como número de comparações, número de trocas e tempo de execução mostrou-se fundamental para compreender as diferenças entre os algoritmos estudados e sua adequação a diferentes cenários computacionais.

Conclui-se que o conhecimento das características, vantagens e limitações de cada algoritmo é essencial para o desenvolvimento de sistemas eficientes, escaláveis e capazes de atender às demandas de processamento de dados presentes na computação moderna.

7 REFERÊNCIAS

- ASCENCIO, Ana Fernanda Gomes; CAMPOS, Edilene Aparecida Veneruchi. *Fundamentos da programação de computadores*. 3. ed. São Paulo: Pearson, 2012.
- CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. *Algoritmos: teoria e prática*. 3. ed. Rio de Janeiro: Elsevier, 2012.

FORBELLONE, André Luiz Villar; EBERSPÄCHER, Henri Frederico. *Lógica de programação: construção de algoritmos e estruturas de dados*. 3. ed. São Paulo: Pearson, 2005.

GOODRICH, Michael T.; TAMASSIA, Roberto. *Estruturas de dados e algoritmos em Java*. 5. ed. Porto Alegre: Bookman, 2013.

SEDGEWICK, Robert; WAYNE, Kevin. *Algorithms*. 4. ed. Boston: Addison-Wesley, 2011.